Lab: Differential Testing Practice in Pharo

Duration: 1 hours **Tool:** Pharo

In this practice, we will learn how to use differential testing as an oracle for fuzzing. Remember, however, that differential testing can be used for any kind of testing!

Differential testing is the process of finding errors by comparing the behaviour of two similar entities (classes, systems, applications). The hypothesis is that both entities satisfy the same specification. You can imagine for example comparing the output of two JSON serializers.

In this practice we will use the phuzzer library on top of Pharo 11.

Setup

For this practice, you will need to:

- install the pharo launcher
- install Pharo11 through the launcher
- install phuzzer in Pharo:
  - open a playground
  - execute the following expression:

```
Metacello new
  baseline: 'Phuzzer';
  repository: 'github://alamvic/phuzzer:main';
  onConflictUseIncoming;
  load.
```

Getting two Parsers for the same specification

Pharo 11 counts with 3 different date parsers. - an unstructured parser that is used through the `String>>asDate` method with, e.g., `'1-2-2003' asDate` - a format-based date parser that uses a format to guide parsing, e.g., `Date readFrom:`

`'1-2-2003' readStream pattern: 'm-d-yyyy'`

We can now fuzz each of these in isolation:

```
"The simple parser"
f := PzRandomFuzzer new.
ra := PzBlockRunner on: [ :e | e asDate ].
ra expectedException: DateError.
f run: ra times: 20.

"The format-based parser"
f := PzRandomFuzzer new.
rb := PzBlockRunner on: [ :e | Date readFrom: e readStream pattern:
'm-d-yyyy' ].
rb expectedException: DateError.
f run: rb times: 1000.
```

Things to think about:

- Do they work the same? Do they show the same issues?
- Do you imagine how we could use the results we had for validation?

The differential runner

`PzBlockRunner` categorises the results as follows: - **an error:** if the block fails with an unexpected exceptions - **a success:** if the block fails with an expected exception or if it does not fail at all

However, with two parsers that follow the same(ish) specification, we can compare both their outputs. If they give the same result, they are either both ok, or both have the same bug. But if they show different results, this is a clear indication of a potential bug in one of them!

We can then run a differential fuzzer that looks for disagreements! Run the code below which fuzzes 1000 times and inspect the results.

```
rd := PzDifferentialRunner new.
rd runnerA: ra.
rd runnerB: rb.
f run: rd times: 1000.
```

Things to think about:

- Check the successes. Are they real successes?
- What about the errors? Are we catching bugs that we did not catch before?
- Check the code of the `PzDifferentialRunner` that looks for agreements: could it be stronger? How could we configure the code that decides if two outcomes are the same with, for example, a strategy pattern? We could for example say that two failures are the same if they are both expected, or both unexpected.

More exercises

1. Repeat the exercises before but now using other kind of fuzzers. Use different random fuzzers, grammars and mutations.

Thinks to think about:

- Are the comparisons fair? Remember that both parsers will parse differently inputs such as `'17-2-2003'`. How can we ensure that we make fair comparisons?
- What could you do if you don't have many alternative implementations? Could you build some? Could you build some from another one? For example, how could we make an alternative implementation for the format `'dd-mm-yyyy'`.

1. Investigate about property-based testing. What kind of properties could a date parser satisfy? How can we build an oracle from that property?

2. Try fuzzing the format and the input string at the same time. If the generated input almost never complies to the format, most inputs will fail. How can you build inputs that almost always comply to the format? Or formats that comply to the inputs?

Further?

From here you can go explore alternative solutions for oracles. The two most prominent being metamorphic relations and invariant generators.